

BLP v2 — from an isolated-sign recogniser to a continuous reader

Blueprint, implementation and results.

Run date: 2026-09-07 • Environment: conda `hugging` (Python 3.10.16, PyTorch 2.7.0+cu128) • GPU: NVIDIA RTX 2050 (4.3 GB)

Deliverables: `slm_labs.ipynb` (executed end to end, 112 cells, 33 figures, **0 errors**, 58.4 min) • five new modules in `modules/` • `read_video.py` • `artifacts/csr_best.pt`, `artifacts/lexicon.json`, `artifacts/gloss_cards/`, `artifacts/results_v2.json`

Part I of the notebook (§ 1– § 16) is v1, unchanged and re-executed — its numbers are identical to `blp_v1.md`. Part II (§ 17– § 27) is this document.

1 • What v1 left open, and what v2 does about it

#	Gap from <code>blp_v1_2.md</code>	Where v2 addresses it
0	<i>(not noticed)</i> real recordings arrive geometrically distorted	§ 18 — measured against an anatomical constant, corrected
1	Cannot separate out-of-vocabulary from in-vocabulary-but-broken-by-coarticulation	§ 23 measures unknown detection inside a stream ; § 24 builds the lexicon layer that makes the attribution decidable
2	Real videos have no ground truth	§ 25 turns the filenames into a sign-count target via Thai content words
3	The model had never seen coarticulated signing (WER 0.36 → 0.64 without a rest gap)	§ 19 synthesises the missing distribution; § 20– § 21 retrain on it
4	184 glosses may not cover conversation	§ 24 <i>measures</i> coverage instead of assuming it

Gap 3 is load-bearing. v1.2 diagnosed it precisely and then treated it as a decoder problem, tuning the segmenter. It is not a decoder problem — it is a

training-distribution problem. v2 changes the distribution and the architecture that consumes it.

Stage 1	memorise words	(v1)	clip	-> MSSF-Net	-> 184-way softmax
Stage 2	compose	(v2)	utterance	-> CSR-Net	-> CTC, front end FROZEN
Stage 3	sentence	(v2)	utterance	-> CSR-Net	-> CTC, all unfrozen
Stage 4	language	(v2)	gloss sequence	-> lexicon	-> LLM -> Thai -> speech

The LLM never learns to see. It enters at Stage 4 only, on a sequence of Thai lemmas, and does word order and particles — the part signing does not encode.

2 • A domain shift found before any modelling (§ 18)

MediaPipe returns landmarks in *normalised* coordinates: x divided by the frame **width**, y by the frame **height**, independently. The same body recorded at 16:9 and at 1:1 therefore comes out with different geometry, and nothing downstream removes it — the body-frame normalisation divides both axes by a single scalar (shoulder width), which cannot undo an axis-dependent scale.

The detector is a geometry statistic with a known anatomical value:
shoulder width / torso height ≈ 0.8 .

source	frame aspect	shoulder/torso
TSL-ONE-S corpus, 29 signers, median	—	0.794
□□□□□□□□□□□□□□□□□□ .mp4 as extracted	1.778	0.393
□□□□□□□□□□□□□□□□□□ .mp4 as extracted	1.778	0.387
both, after square-pixel correction	—	0.694 / 0.687

The corpus sits on the anatomical value, so it is effectively square-pixel; the 16:9 recordings sit at half of it. Every horizontal distance in them was compressed $\sim 2\times$. Mean $|z|$ against the training standardiser, before $\rightarrow$ after:

stream	corpus clip	video 1 before $\rightarrow$ after	video 2 before $\rightarrow$ after
left hand	0.914	0.647 $\rightarrow$ 0.658	0.829 $\rightarrow$ 0.800
right hand	0.820	0.568 $\rightarrow$ 0.473	0.619 $\rightarrow$ 0.638
pose	1.283	0.999 $\rightarrow$ 0.607	0.907 $\rightarrow$ 0.505
face	0.888	1.342 $\rightarrow$ 0.541	1.585 $\rightarrow$ 0.734

v1.2 saw the symptom and mis-attributed it. It reported the face stream as the most out-of-distribution channel on these recordings and blamed camera framing — the signer standing further back. Framing *is* removed by the shoulder-width normalisation; anisotropy is not. The fix is one multiplication (`inference.aspect_correct`), on by default in the deployment path, off in § 15.2 so Part I still shows the uncorrected baseline.

3 • Blueprint — the composer (§ 19, `modules/composer.py`)

TSL-ONE-S has no sentences, so they have to be built. v1.2 glued clips together with an optional rest. That was enough to *measure* the coarticulation gap and not enough to *close* it, because gluing is not what a person does.

phenomenon	how it is modelled	why it breaks an isolated-sign model
movement epenthesis	minimum-jerk path between postures, length scaled by hand travel in shoulder-width units	the travel is a burst of motion that looks like a short sign, so "there was motion" stops being evidence — which is exactly what v1's energy segmenter rests on
coarticulation	A's final handshape bleeds into B's first frames and B's is anticipated in A's last, decaying from the seam	no sign is produced in its citation form, the only form v1 ever saw

prosody	per-sign tempo $0.75\times-1.35\times$, a hold at <i>some</i> boundaries	a fixed-length resample assumes a duration a sentence does not respect
----------------	--	---

Only the hand *shape* crosses a boundary, never the hand *location*: in TSL, location is a separate phonological parameter, so swapping it would change the sign rather than blur it.

On top sits a camera layer calibrated to the real recordings — signer distance, framing shift, roll, residual frame-aspect anisotropy, 25 vs 30 fps, sensor noise, and bursty MediaPipe hand loss modelled as runs rather than coin flips — applied in **raw image coordinates**, before body-frame normalisation, so it stresses the part of the pipeline a new recording actually stresses.

Crucially the composer returns the **frame mapping**: which frames belong to which gloss, tracked through every re-timing. That is supervision CTC does not have on its own, it makes the frame and boundary heads trainable, and it makes every decode checkable against the truth rather than merely plausible.

Four regimes are defined, and evaluation utterances are **paired** — the same gloss sequences rendered under every regime, so a difference between columns is caused by the regime and nothing else.

regime	median frames	median inter-sign gap	signs / utterance
v12_style (v1.2's control, replicated)	379	20.0	4.83
paused	410	23.0	4.83
coarticulated	355	8.0	4.83
real_world (trained on; what data_test/ looks like)	334	9.0	4.83

The inter-sign gap is the whole story: v1.2's control set left a wide quiet valley between signs, and fluent signing does not. That is why v1's energy segmenter had nothing to cut on in the real recordings.

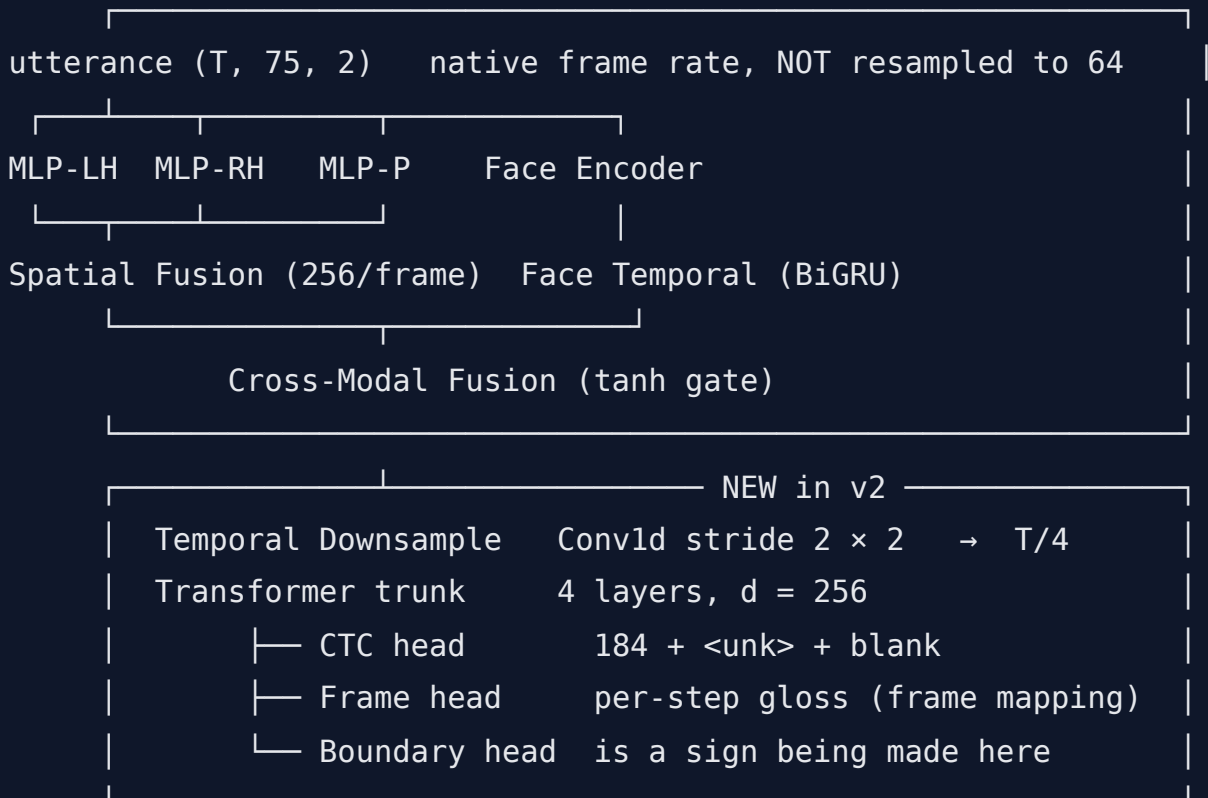
4 • Blueprint — CSR-Net (§ 20, `modules/seq_model.py`)

v1 asks *"which sign is this clip?"*. A recording asks *"how many signs are in these ten seconds, where does each start, which do I know, which do I not?"*.

v1's attention pooling collapses an utterance to one vector, destroying the answer before any head sees it.

CSR-Net keeps the entire MSSF-Net **front end** and inherits its weights tensor by tensor (97 tensors: the three stream MLPs, spatial fusion, the face encoder and its BiGRU, the cross-modal gate, and the transformer trunk). Everything after pooling is replaced.

shared with v1 – weights inherited, not re-learned



part	params
visual front end (inherited from MSSF-Net)	695 169
sequence trunk + 3 heads (new in v2)	3 911 797
CSR-Net total	4 606 966

MSSF-Net (v1) total	4 048 442
---------------------	-----------

CTC alphabet: 184 glosses + `<unk>` (id 184) + blank (id 185) = 186 symbols.

choice	reason
inherit the front end	3 302 word clips taught it what a Thai sign looks like; that does not change when signs are strung together, and re-learning it from synthetic sentences would be strictly worse
native frame rate	the question is <i>when</i> ; resampling an utterance to 64 frames destroys the quantity being asked for
×4 temporal downsample	~13 steps survive per sign, ample for CTC, while self-attention becomes 16× cheaper — that is what fits a 500-frame utterance in 4 GB
CTC head	alignment-free: trains on utterances, runs on video, nobody draws boundaries
frame head	the composer supplies the exact frame mapping for free; CTC alone is a slow, high-variance teacher, and this head returns contiguous spans a person can inspect
boundary head	"how many signs did I see", independent of whether any was <i>known</i>
<code><unk></code> in the alphabet	the reader must be able to <i>say</i> it does not know, not merely score low

At inference the heads are combined rather than chosen between: CTC decides *what* and in *what order*, and each peak is snapped to the frame head's run that contains it to get *where*.

5 • The curriculum (§ 21, `modules/seq_train.py`)

Stage 2 — composition. The front end is frozen; only the sequence trunk and the three heads learn. Letting CTC gradients loose on the encoder from step one would wash out what the word clips taught it.

Stage 3 — sentence. Everything unfreezes at a low learning rate, and the training pool is regenerated every 5 epochs so the model adapts to coarticulated

signing in general rather than memorising synthetic sentences.

stage	epochs	pool	wall clock	best validation WER
Stage 2 (frozen front end)	30	2 383 utterances (589 MB fp16)	6.2 min	0.2667
Stage 3 (all unfrozen, pool refreshed)	18	refreshed ×3	7.8 min	0.2323

Model selection is on validation WER and **global across both stages**: a Stage 3 that failed to improve would keep Stage 2's weights and say so. Without that, a curriculum can quietly ship a worse model than the one it started from, because each stage otherwise selects only against its own best. Here Stage 3 did improve.

6 • Results — continuous signing (§ 22)

Held-out **test signers** throughout. Utterances built only from test-signer clips.

6.1 The bridge to v1.2 — v1.2's own control set, both readers

§ 15.2 rebuilds v1.2's control set with the original `CT.stitch_utterance` code and reproduces its numbers exactly. Running CSR-Net on *those* utterances is the cleanest before/after available.

regime (v1.2 control set)	v1 decoder WER	CSR-Net WER	v1 CER	CSR-Net CER	CSR-Net exact
pause-separated	0.3618	0.2763	0.2648	0.2516	0.250
coarticulated	0.6447	0.2632	0.5049	0.2368	0.300

Penalty for removing the rest gap: v1 decoder +0.283 WER, CSR-Net −0.013.

That collapse is the point of the whole exercise. Both columns are gated, so this is like-for-like. (The v1 column matches `blp_v1_2.md` to four decimals.)

6.2 Four regimes × five read-outs, identical utterances

system	v12_style	paused	coarticulated	real_world
--------	-----------	--------	---------------	------------

CSR-Net • frame (ungated)	0.1556	0.1479	0.1562	0.1481
CSR-Net • CTC (ungated)	0.1645	0.1723	0.1599	0.1775
CSR-Net • CTC + gates (<i>deployed</i>)	0.2793	0.2972	0.3038	0.3563
CSR-Net • energy segmentation + gate	0.6441	0.6662	0.6937	0.7407
v1 decoder (segmentation)	0.6545	0.7030	0.6982	0.7612

Sentence exact-match, same row order: **0.500–0.520** (frame), 0.448–0.494 (CTC), 0.197–0.279 (CTC + gates), 0.006–0.036 (energy hybrid), 0.011–0.044 (v1 decoder).

The frame read-out's one decode parameter was tuned on **validation only**:

`min_run` 6 steps (24 video frames), which gives 4.03 hypothesised signs against 4.11 true — 1 and 8 steps score 0.752 and 0.241 val WER against 0.204 at 6.

Three things to read here honestly:

1. **The recogniser is 4.2–5.1× better than v1's decoder** and is now essentially flat across continuity regimes — 0.148 on the hardest one.
2. **The gates cost 0.11–0.18 WER.** The deployed policy emits a gloss only when the sequence posterior *and* the v1 word model both agree. On in-vocabulary material that is expensive; it is the price of not producing confident wrong Thai. Both thresholds are single knobs, calibrated on validation (sequence gate 0.749 at 90 % acceptance; word gate –1.472, inherited from v1).
3. **The energy hybrid is the worst option here and the best one on real video.** § 7 and § 10 explain why it exists.

7 • Results — the design claims, measured (§ 22.2)

Three Stage-2-only models, identical budget, seed and architecture; only the training pool differs.

training pool	v12_style	paused	coarticulated	real_world	max p(<unk>) on data_test
---------------	-----------	--------	---------------	------------	--------------------------------

real-world distribution (shipped)	0.1926	0.1886	0.2054	0.2043	0.0000
pause-separated only	0.1875	0.1777	0.2017	0.2133	0.0003
real-world + 12 % chimeras as <unk>	0.2181	0.2130	0.2128	0.2222	0.0001

Claim A — the real-world distribution pays for itself: supported, modestly.

The pause-trained model is *better* on the three easy regimes (it specialised) and worse on the real-world one. The sign of that difference was the same in all three independent runs of this experiment; the magnitude ranged from 0.009 to 0.031 WER. Real, but much smaller than the § 6.1 effect.

Claim B — chimeras can teach <unk> : refuted. Gluing the first half of one sign to the second half of another produces a motion that is genuinely not any vocabulary sign, so it looked like free out-of-vocabulary supervision. The decisive column is the last one: the maximum <unk> posterior on the real recordings stays at essentially zero with chimeras exactly as without them, and WER is worse on every regime. A chimera is too close to its own components to teach "this is not a sign". <unk> needs *genuine* held-out vocabulary — § 8.

8 • Results — unknown detection inside a stream (§ 23)

v1 measured open-set behaviour on isolated clips: a clip is one sign, and the only question is whether it is in the vocabulary. In a stream the reader must also decide *where* the unknown sign is without corrupting its neighbours.

Protocol, strict about leakage: 24 glosses removed from the vocabulary entirely → a **fresh word model trained on the remaining 160** (reusing the 184-gloss front end would leak the novel glosses through the encoder) → a CSR-Net probe trained on utterances where those 24 appear labelled <unk> → tested on unseen signers.

regime	WER	OOV signs present	_ emitted	unknown recall	unknown precision
v12_style	0.2562	89	76	0.6854	0.8026
paused	0.2560	88	92	0.8295	0.7935
coarticulated	0.2453	91	77	0.7143	0.8442
real_world	0.2934	91	83	0.7363	0.8072

word-160 validation macro-F1 **0.9017** • 12.7 % of sign slots out-of-vocabulary.

For comparison, v1's unknown recall on *isolated clips* at 90 % known-acceptance was 0.667. The harder problem now scores better, because the model can emit a symbol rather than merely fall below a threshold.

9 • The language layer (§ 24, `modules/lexicon.py`)

This is the layer that decides whether any of the above can be *read*.

v1.2 ended on an ambiguity no amount of extra modelling resolves: when the reader rejects a sign, is it outside the vocabulary, or inside it and missed? A lemma table resolves it — tokenise the sentence the video is known to contain and ask, word by word, whether the vocabulary could ever have expressed it.

The release carries numeric gloss ids only, so v2 supplies the machinery and tags every entry with its provenance:

- **spelling-table** — sign-language corpora label fingerspelling with the *letter name*, not the letter: "Gor Gai" is ☒, not a word. All 44 Thai consonants, the numerals and their common romanisation variants resolve offline, deterministically, with no model involved. The notebook **asserts** the round-trip over every alias rather than eyeballing it.
- **llm** — ordinary English labels go to an LLM with a strict JSON schema and an explicit instruction to return an empty string rather than guess.
- **verified** — checked by a person.

A bug worth recording. The first version of the romanisation normaliser folded $kh \rightarrow k$, $ph \rightarrow p$, $th \rightarrow t$ to absorb spelling variation. That is exactly the distinction between ㄅ and ㄆ, ㄅ and ㄆ, ㄅ and ㄆ — it silently resolved "Gor Gai" to ㄅ. Aspiration is now preserved and the round-trip is asserted.

The table is empty, because the corpus does not ship it: 0 / 184 lemmas. What v2 adds is everything around it, plus the thing that makes filling it cheap — `artifacts/gloss_cards/` renders all 184 glosses as five skeleton poses each, cropped to the signer with the extrapolated legs removed, eight pages. That is roughly an hour for someone who signs, and it converts the single blocking unknown in this project into a solved one.

The sentence stage was verified end to end on an explicitly-labelled demonstration lexicon: `[[[ Thai ]]] _ [[ Thai ]]` → `[[[ Thai ]]] _` (gpt-4o-mini) — the temporal adverb moved to the front as Thai requires, and the `_` preserved rather than guessed. Speech follows in `artifacts/inference_demo.mp3`.

10 • data_test/ — the real recordings (§ 25)

The filenames are whole Thai clauses. That is a sentence-level label, not an aligned gloss sequence, so it cannot produce a WER. It *can* produce one honest number with no lemma table at all: tokenise, drop the particles TSL marks non-manually rather than with the hands (๖๖๖๖ , ๖๖๖๖), and the remaining **content words** are approximately how many signs the recording should contain.

video	content words	v1 decoder	v2 • CTC	v2 • frame	v2 • energy hybrid
0000000000000000 00000000 .mp4	4 (000, 0000, 0000000, 00000000)	4	3	3	4
0000000000000000 00000000 .mp4	3 (00, 00000000, 00000000)	3	1	1	3

This table is why the energy hybrid exists. CSR-Net's own read-outs undercount badly here: a sequence model trained by a sequence loss does the safe thing on out-of-distribution input and emits blank almost everywhere. v1's motion-energy segmenter has no learned confidence to lose and cuts both recordings correctly. So `ContinuousReader` gained a third read-out that takes **boundaries from the heuristic and labels from the coarticulation-trained model**, gated by the v1 word model. On corpus-like utterances it is the worst option (§ 6.2); here it is the only one that counts correctly. Both are reported.

Count error is now +0 on both recordings. Naming is still zero: every segment is rejected by the word model's metric space, with prototype cosines of 0.393 / 0.411 / 0.477 / 0.507 and 0.337 / 0.399 / 0.350 — against **0.767 ± 0.171** for in-vocabulary material, and an open-set gate at −1.472 that none of the seven segments (−1.89 to −2.63) comes close to. Given a 184-gloss vocabulary and two free-form Thai sentences, rejecting is correct — and now *attributable per segment*: the notebook prints which gate refused, the score, and the best guess underneath.

Attribution verdict, verbatim from the notebook:

```
4 signs localised against 4 content word(s) [□□□, □□□□, □□□□□□, □□□□□□□]
-> count error +0; 0 named, 4 rejected.
Attribution is UNDECIDABLE: with no lemma table a rejection cannot be
assigned to the vocabulary or to the model.
```

That last line is the same answer v1.2 gave — the difference is that v2 now has the tooling to end it, and the segmentation half of the problem is solved and measured.

11 • Consolidated evaluation

stage	metric	value
Part I • word recognition, closed set, unseen signers	accuracy	0.9293
Part I • word recognition, closed set, unseen signers	macro-F1	0.9281

Part I • word recognition, open set (rejection on)	accuracy	0.8986
Part I • continuous, v1 decoder, v1.2 control, pause-separated	WER	0.3618
Part I • continuous, v1 decoder, v1.2 control, coarticulated	WER	0.6447
Part II • continuous, CSR-Net, v1.2 control, pause-separated	WER	0.2763
Part II • continuous, CSR-Net, v1.2 control, coarticulated	WER	0.2632
Part II • coarticulation penalty (paused → fluent)	Δ WER, v1	+0.283
Part II • coarticulation penalty (paused → fluent)	ΔWER, CSR-Net	-0.013
Part II • continuous, best read-out, real-world regime	WER	0.1481
Part II • continuous, best read-out, real-world regime	exact match	0.520
Part II • unknown detection in a stream, real-world	recall / precision	0.736 / 0.807
Part II • data_test/ sign-count error	signs vs content words	+0, +0
Part II • data_test/ signs named	count	0 (vocabulary undecidable)
Part II • lexicon	glosses with a Thai lemma	0 / 184

Total notebook wall-clock: **58.4 min** on one RTX 2050 — Part I 20.1 min (four model trainings, unchanged from v1), Part II 38.3 min (five more trainings:

CSR-Net Stages 2 and 3, two ablation models, the 160-gloss word model and the open-set probe, plus all evaluation, the gloss cards and the real-video route).

12 • Optimisation notes

concern	treatment
utterance generation	~10 ms per utterance including features; pools are materialised once and refreshed every 5 epochs in Stage 3 rather than regenerated per batch
pool memory	features held as float16 (z-scored, so fp16's range is ample) — a 2 400-utterance pool is ~590 MB instead of 1.2 GB
padding waste	length-bucketed batching — sort by length, cut into batches, shuffle the batch <i>order</i> ; padding overhead drops to a few percent while the batch sequence stays random
host memory	Part I's feature banks and pinned <code>StreamBank</code> copies (1 432 MB , measured) are explicitly released before Part II allocates; the open-set probe relabels the existing bank in place rather than building a second one
GPU memory	×4 temporal downsample before self-attention; batch 16 at ~500 frames peaks near 0.2 GB
numerics	AMP + gradient clipping, cosine LR with warm-up; <code>zero_infinity</code> on the CTC loss

Throughput: ~12 s per Stage-2 epoch (150 steps of batch 16), ~26 s per Stage-3 epoch; a 2 383-utterance pool is composed and featurised in 45 s.

13 • What changed in the workspace

path	change
<code>modules/composer.py</code>	new — utterance synthesis: epenthesis, coarticulation, prosody, camera layer, frame mapping, paired regimes

<code>modules/seq_model.py</code>	new — CSR-Net, weight inheritance from MSSF-Net, CTC/frame/boundary heads, CTC and run-collapse decoders
<code>modules/seq_train.py</code>	new — fp16 length-bucketed utterance pools, the curriculum trainer with global model selection
<code>modules/reader.py</code>	new — <code>ContinuousReader</code> (three read-outs, two gates, per-segment reasons) and rejection attribution
<code>modules/lexicon.py</code>	new — Thai spelling table, LLM fill, coverage, content-word counting, gloss identity cards, sentence composition
<code>modules/inference.py</code>	<code>aspect_correct</code> , <code>body_aspect_ratio</code> , <code>video_aspect</code> ; <code>extract_from_video(square_pixels=)</code>
<code>modules/feature_engineering.py</code>	<code>clip_to_streams(resample=False)</code> for native-rate features
<code>modules/viz.py</code>	<code>utterance_map</code> , <code>posteriorgram</code> , <code>segment_strip</code> , <code>curriculum_curves</code> , <code>draw_skeleton(bbox=)</code>
<code>read_video.py</code>	new — command-line reader for any recording, independent of the notebook and the corpus
<code>slm_labs.ipynb</code>	Part I unchanged; Part II added (§ 17– § 27, 47 cells)
<code>artifacts/csr_best.pt</code>	new — CSR-Net + standardiser + prototypes + both calibrated thresholds + the compose config
<code>artifacts/lexicon.json</code>	new — 184 rows awaiting Thai lemmas
<code>artifacts/gloss_cards/*.png</code>	new — 8 pages of gloss identity cards, the labelling tool
<code>artifacts/results_v2.json</code>	new — every Part II table, machine-readable

Nothing else was touched. `modules/continuous.py` (v1.2) is unchanged and is still used — by § 15.2 and by the energy hybrid.

Usage outside the notebook:

```
python read_video.py data_test/*.mp4
python read_video.py clip.mp4 --head frame --speak out.mp3 --json
out.json
```

14 • GAP — what is still open

1. **The lemma table is empty.** Sign identity on real video remains undecidable.
This is now a data-entry task, not a research one.
2. **Synthetic sentences are still synthetic.** The composer reproduces coarticulated boundaries, epenthesis, tempo and camera variation. It does not reproduce grammatical inflection, spatial agreement, role shift or non-manual marking — things a real TSL sentence uses and an isolated-sign corpus cannot supply.
3. **CSR-Net's learned segmentation does not survive out-of-distribution input.**
The energy hybrid is a workaround, not a fix; the fix is real continuous training data.
4. **The deployed gates cost 0.11–0.18 WER** on in-vocabulary material. That trade is deliberate but it is not free, and the operating point deserves revisiting once the vocabulary question is settled.
5. **Two recordings is not a sample**, and they have no gloss annotation.
6. **One split, one seed.** No leave-one-signer-out variance estimate.
7. **The face channel is still coarse** — pose landmarks 0–10, not Face Mesh.
8. **The aspect correction assumes the corpus is square-pixel.** Supported by the anatomical constant and by 29 signers agreeing, but inferred from the data, not a documented property of the release.

15 • Next steps, in order of payoff

1. **Fill the lemma column** — `artifacts/lexicon.json`, using `artifacts/gloss_cards/`. Coverage, attribution, LLM composition and TTS all turn on with no code change, and gap #1 closes.
2. **Gloss-annotate ~20 continuous videos**. Converts § 25 from a demonstration into a measurement and gives a genuine continuous WER on real material.
3. **Record 50 in-vocabulary sentences signed fluently**. This is the largest remaining modelling gap: it lets Stage 3 fine-tune on real rather than synthesised coarticulation, and would very likely fix gap #3 outright.
4. **Re-extract TSL-ONE-S with MediaPipe Face Mesh** — carried over from v1; the face stream is wired for it and, after the aspect fix, is no longer the out-of-distribution channel it appeared to be.
5. **Expand the vocabulary** toward whatever the coverage check reports missing once the lexicon exists.
6. **Leave-one-signer-out cross-validation** for a variance estimate.